Lecture 30.02: Gradient Boosting Classifier (Intuition) — Log-Odds & Sigmoid Residuals







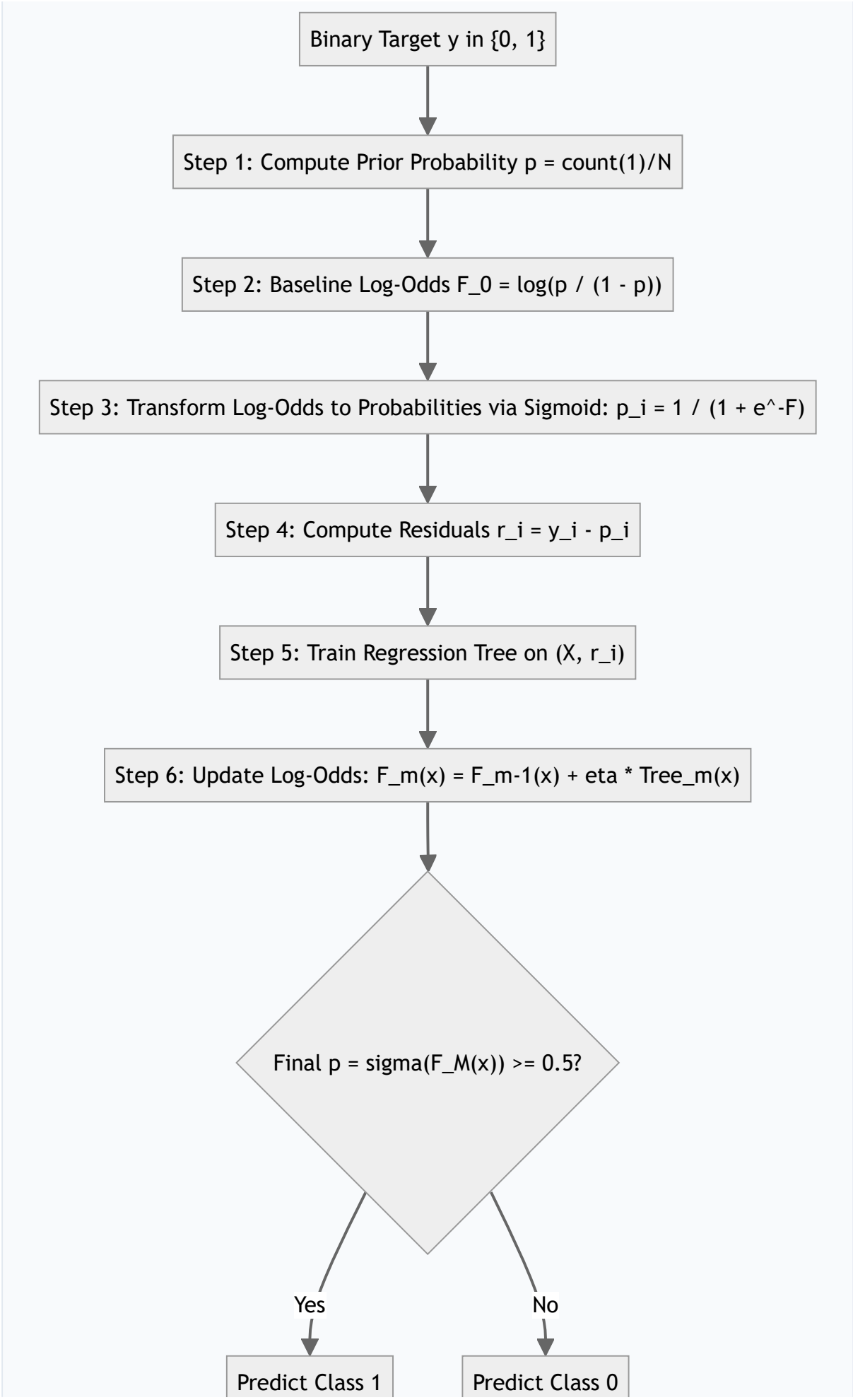




Topic Overview

Gradient Boosting for binary classification maps targets y in {0, 1} into continuous log-odds space. The initial prediction F_0 represents the prior log-odds of the positive class. Probabilities are derived via the logistic sigmoid transform p = sigma(F), yielding pseudo-residuals r_i = y_i - p_i. Shallow decision trees model these residuals, updating log-odds iteratively before final probability thresholding.

Visual Architecture Diagram


Binary Target y in {0, 1}
Step 1: Compute Prior Probability p = count(1)/N
Step 2: Baseline Log-Odds F_0 = log(p / (1 - p))
Step 3: Transform Log-Odds to Probabilities via Sigmoid: p_i = 1 / (1 + e^-F)
Step 4: Compute Residuals r_i = y_i - p_i
Step 5: Train Regression Tree on (X, r_i)
Step 6: Update Log-Odds: F_m(x) = F_m-1(x) + eta * Tree_m(x)
Final p = sigma(F_M(x)) >= 0.5?
Yes
No
Predict Class 1
Predict Class 0

🛠️ Mathematical Mechanics & Theoretical Foundations

For binary cross-entropy (log-loss) $L(y, F) = -y \ln(p) - (1-y)\ln(1-p)$ where $p = \sigma(F) = \frac{1}{1+e^{-F}}$:

The negative gradient with respect to log-odds F simplifies elegantly to:

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right] = y_i - p_i$$

The initial baseline log-odds F_0 from training class prior p:

$$F_0 = \ln\left(\frac{p}{1-p}\right) = \ln\left(\frac{\sum_{i=1}^{N} y_i}{N - \sum_{i=1}^{N} y_i}\right)$$

The terminal leaf output γ_{jm} for region R_{jm} in a tree maximizing the second-order Taylor expansion:

$$\gamma_{jm} = \frac{\sum_{x_i \in R_{jm}} r_{im}}{\sum_{x_i \in R_{jm}} p_{im}(1-p_{im})}$$

💻 Python Production Implementation

```
import numpy as np

def sigmoid(z):
    """Numerically stable logistic sigmoid function."""
    return np.where(z >= 0, 1.0 / (1.0 + np.exp(-z)), np.exp(z) / (1.0 + np.exp(z)

def gradient_boosting_binary_classifier_scratch(X, y, n_estimators=5, learning_rat
    """
    First-principles demonstration of Gradient Tree Boosting for Binary Classifica
    Operates in Log-Odds space using the logistic sigmoid loss.
    """
    X_arr = np.asarray(X, dtype=float)
    y_arr = np.asarray(y, dtype=float)
```

```
    # 1. Baseline prior probability and log-odds F0
    p_prior = np.mean(y_arr)
    # Clip to avoid division by zero
    p_prior = np.clip(p_prior, 1e-6, 1.0 - 1e-6)
    f0 = float(np.log(p_prior / (1.0 - p_prior)))

    current_log_odds = np.full_like(y_arr, fill_value=f0)

    # 2. Sequential Boosting Stages
    from sklearn.tree import DecisionTreeRegressor
    trees = []

    for stage in range(n_estimators):
        # Convert current log-odds to probability
        current_probs = sigmoid(current_log_odds)

        # Residuals = y - p
        residuals = y_arr - current_probs

        # Fit regression tree on pseudo-residuals
        tree = DecisionTreeRegressor(max_depth=1, random_state=42 + stage)
        tree.fit(X_arr, residuals)
        trees.append(tree)

        # Update log-odds via shrinkage
        current_log_odds += learning_rate * tree.predict(X_arr)

    final_probs = sigmoid(current_log_odds)
    predicted_classes = (final_probs >= 0.5).astype(int)

    return f0, trees, final_probs, predicted_classes

# ----------------- Driver Execution & Testing -----------------
if __name__ == "__main__":
    X_sample = np.array([
        [22.0, 25.0], [45.0, 55.0], [52.0, 70.0],
        [20.0, 22.0], [38.0, 48.0], [60.0, 85.0]
    ])
    y_sample = np.array([0, 1, 1, 0, 0, 1])  # 0 = Low Risk, 1 = High Risk

    f0_init, trained_trees, probas, preds = gradient_boosting_binary_classifier_sc
        X_sample, y_sample, n_estimators=6, learning_rate=0.3
    )

    print("=" * 60)
    print("--- GRADIENT BOOSTING CLASSIFIER (LOG-ODDS SCRATCH) ---")
    print(f"Initial Baseline Log-Odds (F0) : {f0_init:.4f}")
    print(f"Actual Target Classes          : {y_sample.tolist()}")
    print(f"Final Model Probabilities      : {[round(p, 3) for p in probas]}")
    print(f"Predicted Discrete Classes     : {preds.tolist()}")
```

```
    print(f"Classification Accuracy        : {np.mean(preds == y_sample) * 100:.1f
    print("=" * 60)
```

🔑 Critical Architectural Takeaways

1. **Sequential Bias Minimization**: Unlike bagging which stabilizes variance, boosting models sequentially reduce bias by focusing on residuals and hard examples.
2. **Shrinkage Regularization**: Setting learning_rate < 1.0 enforces small gradient descent steps, preventing the ensemble from memorizing noise.
3. **Weak Learners Principle**: Keeping base decision trees shallow (max_depth=1 to 3) ensures individual estimators remain high-bias weak learners.